Solution: Valid Palindrome III

Problem Statement

Given a string `s` and integer `k`, return `true` if string can be converted into the `palindromic` string after removing at most `k` characters. Otherwise, return `false`.

Examples

- **Example 1**:
    - **Input**: `s = "rdar", k = 1`
    - **Expected Output**: `true`
    - **Justification**: We can make `rdar` to palindromic by removing either `d` or `a` character.
- **Example 2**:
    - **Input**: `s = "abracadabra", k = 5`
    - **Expected Output**: `true`
    - **Justification**: By removing the characters 'b', 'c', 'd', and 'b' , the string becomes "araaara", which is a palindrome. We've removed exactly 5 characters, so the output is `true`.
- **Example 3**:
    - **Input**: `s = "data", k = 0`
    - **Expected Output**: `false`
    - **Justification**: It's impossible to form a palindrome by removing zero characters from "data", so the output is `false`.

Solution

To solve this problem, we will use dynamic programming to keep track of the minimum number of characters that need to be removed to make any substring a palindrome. The core idea is to compare characters from both ends of the string and move towards the center, while keeping track of the minimum deletions needed at each step. This approach works because a palindrome reads the same forwards and backwards, so we can strategically remove characters that disrupt this symmetry. By building up solutions for smaller substrings and using these to solve larger substrings, we ensure efficiency and avoid redundant calculations. The use of dynamic programming is effective here because it breaks down the larger problem into manageable sub-problems, each of which is solved only once.

Step-by-step Algorithm

- Create a 2D array `dp` where `dp[i][j]` represents the minimum number of deletions needed to make the substring from index `i` to `j` a palindrome.
- Initialize `dp[i][i]` to 0 for all `i`, as a single character is always a palindrome.
- For substrings of length 2 to `n` (where `n` is the length of the string):
    - For each starting index `i`, calculate the ending index `j = i + length - 1`.
    - If the characters at `i` and `j` are the same, set `dp[i][j]` to `dp[i + 1][j - 1]` (no additional deletions needed).
    - If they are different, set `dp[i][j]` to the minimum of `dp[i + 1][j]` and `dp[i][j - 1]` plus 1 (delete either `i` or `j` character).
- Check if `dp[0][n - 1]` is less than or equal to `k`. If yes, return `true`, else return `false`.

Algorithm Walkthrough 1

Let's consider the input s = `"data"`, k = `0`

1. **Initialization**:
    - `s = "data"` has a length of `n = 4`.
    - Initialize a 2D array `dp` of size `4x4` with all values set to `0`.
2. **Filling the DP Array**:
    - The `dp` array is filled such that `dp[i][j]` represents the minimum number of deletions needed to make the substring from index `i` to `j` a palindrome.
    - Initially, all `dp[i][i]` are `0` since a single character is always a palindrome.
3. **Iterating Over Substrings**:
    - We consider substrings of increasing lengths. For each substring length (from 2 to `n`), we check every possible substring of that length and update the `dp` array accordingly.
4. **Filling `dp` for `s = "data"`**:
    - Substring Length `2`:
        - Compare `s[0]` (`'d'`) with `s[1]` (`'a'`): Since they are different, `dp[0][1] = 1 + min(dp[0][0], dp[1][1]) = 1`.
        - Compare `s[1]` (`'a'`) with `s[2]` (`'t'`): Since they are different, `dp[1][2] = 1 + min(dp[1][1], dp[2][2]) = 1`.
        - Compare `s[2]` (`'t'`) with `s[3]` (`'a'`): Since they are different, `dp[2][3] = 1 + min(dp[2][2], dp[3][3]) = 1`.
    - Substring Length `3`:
        - Compare `s[0]` with `s[2]`: Since they differ, `dp[0][2] = 1 + min(dp[0][1], dp[1][2]) = 2`.
        - Compare `s[1]` with `s[3]`: Since they are the same, `dp[1][3] = dp[1 + 1][3 - 1] = dp[2][2] = 0`.
    - Substring Length `4`:
        - Compare `s[0]` with `s[3]`: Since they are different, `dp[0][3] = 1 + min(dp[0][2], dp[1][3]) = 1`.
5. **Final DP Matrix**:
    - After filling in the values, the `dp` matrix looks like this:

      ```
      0 1 2 1
      0 0 1 0
      0 0 0 1
      0 0 0 0
      ```
6. **Checking the Result**:
    - We need to check if `dp[0][n - 1]` (which is `dp[0][3]` in this case) is less than or equal to `k`.
    - `dp[0][3]` is `1`, which is not less than or equal to `k` (`0` in this case).
    - Therefore, the final result is `false`: it is not possible to turn `s = "data"` into a palindrome without deletions, as per the `k = 0` constraint.

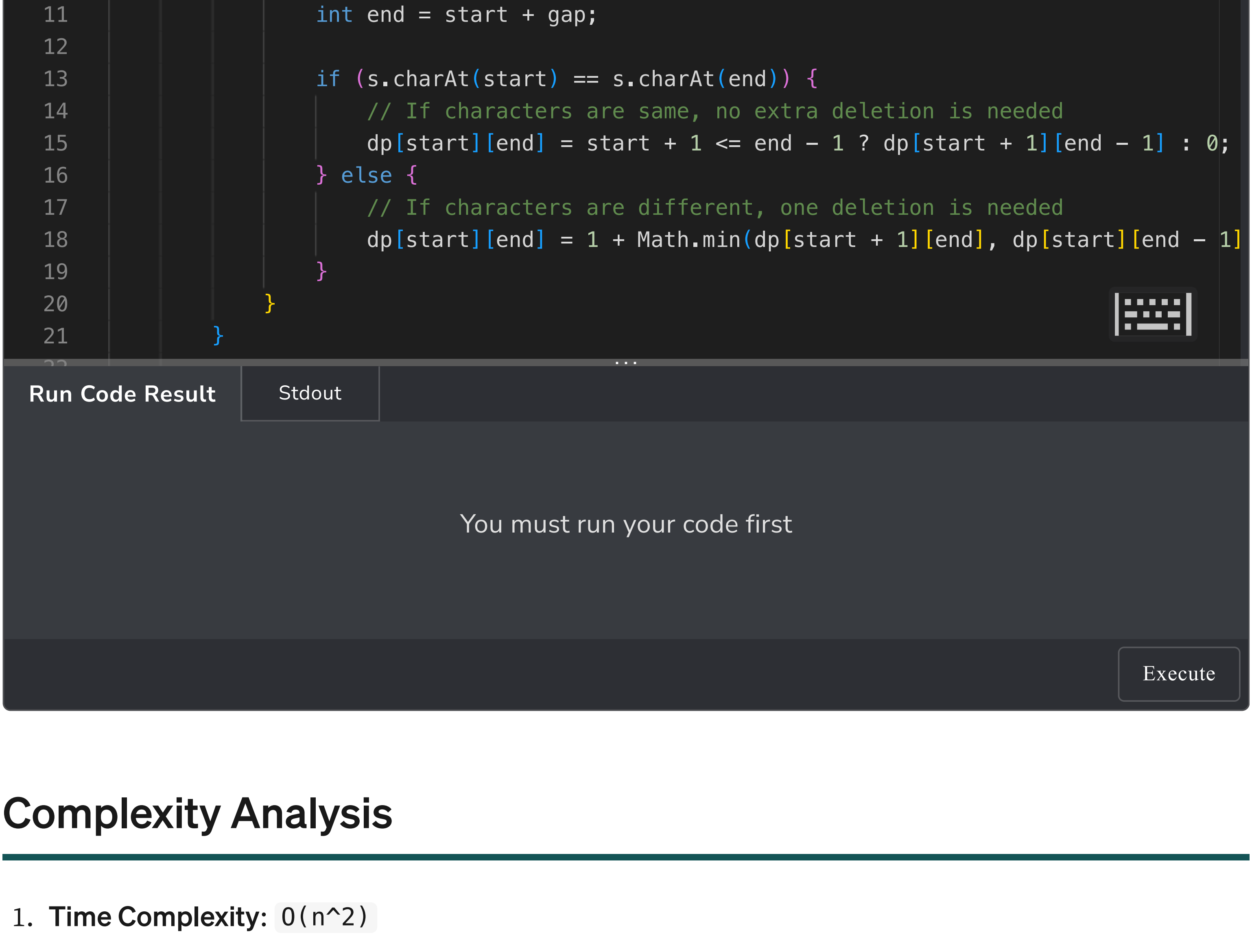
Algorithm Walkthrough 2

Let's consider the input `s = "rdar"` and `k = 1`.

1. **Initialization**:
    - `s = "rdar"` has a length of `n = 4`.
    - Initialize a 2D array `dp` of size `4x4` with all values set to `0`.
2. **Filling the DP Array**:
    - The `dp` array is filled such that `dp[i][j]` represents the minimum number of deletions needed to make the substring from index `i` to `j` a palindrome.
    - Initially, all `dp[i][i]` are `0` since a single character is always a palindrome.
3. **Iterating Over Substrings**:
    - We consider substrings of increasing lengths. For each substring length (from 2 to `n`), we check every possible substring of that length and update the `dp` array accordingly.
4. **Filling `dp` for `s = "rdar"`**:
    - Substring Length `2`:
        - Compare `s[0]` (`'r'`) with `s[1]` (`'d'`): Since they are different, `dp[0][1] = 1 + min(dp[0][0], dp[1][1]) = 1`.
        - Compare `s[1]` (`'d'`) with `s[2]` (`'a'`): Since they are different, `dp[1][2] = 1 + min(dp[1][1], dp[2][2]) = 1`.
        - Compare `s[2]` (`'a'`) with `s[3]` (`'r'`): Since they are different, `dp[2][3] = 1 + min(dp[2][2], dp[3][3]) = 1`.
    - Substring Length `3`:
        - Compare `s[0]` with `s[2]`: Since they are different, `dp[0][2] = 1 + min(dp[0][1], dp[1][2]) = 2`.
        - Compare `s[1]` with `s[3]`: Since they are different, `dp[1][3] = 1 + min(dp[1][2], dp[2][3]) = 2`.
    - Substring Length `4`:
        - Compare `s[0]` with `s[3]`: Since they are the same, `dp[0][3] = dp[1][2] = 1`. (The characters at the beginning and end match, so we check the inner substring "da").
5. **Final DP Matrix**:
    - After filling in the values, the `dp` matrix looks like this:

      ```
      0 1 2 1
      0 0 1 2
      0 0 0 1
      0 0 0 0
      ```
6. **Checking the Result**:
    - We need to check if `dp[0][n - 1]` (which is `dp[0][3]` in this case) is less than or equal to `k`.
    - `dp[0][3]` is `1`, which is equal to `k` (`1` in this case).
    - Therefore, the final result is `true`: it is possible to turn `s = "rdar"` into a palindrome by making at most `1` deletion.

Code



```java
public class Solution {
    public boolean isValidPalindrome(String s, int k) {
        int n = s.length();
        // dp[i][j] will hold the minimum number of deletions required
        // to make the substring s[i...j] a palindrome
        int[][] dp = new int[n][n];

        // Filling the dp array for substrings of increasing lengths
        for (int gap = 1; gap < n; gap++) {
            for (int start = 0; start < n - gap; start++) {
                int end = start + gap;

                if (s.charAt(start) == s.charAt(end)) {
                    // If characters are same, no extra deletion is needed
                    dp[start][end] = start + 1 <= end - 1 ? dp[start + 1][end - 1] : 0;
                } else {
                    // If characters are different, one deletion is needed
                    dp[start][end] = 1 + Math.min(dp[start + 1][end], dp[start][end - 1]
                }
            }
        }
```

You must run your code first

Complexity Analysis

1. **Time Complexity:** `O(n^2)`
    - The algorithm uses a nested loop structure where both the outer and inner loops iterate over the string length `n`. This results in a quadratic time complexity as each character is compared with others in different combinations.
2. **Space Complexity:** `O(n^2)`
    - A 2D array `dp` of size `n x n` is used for dynamic programming, where `n` is the length of the string.

← Previous
Valid Palindrome III (Hard)
☐ Mark as Completed